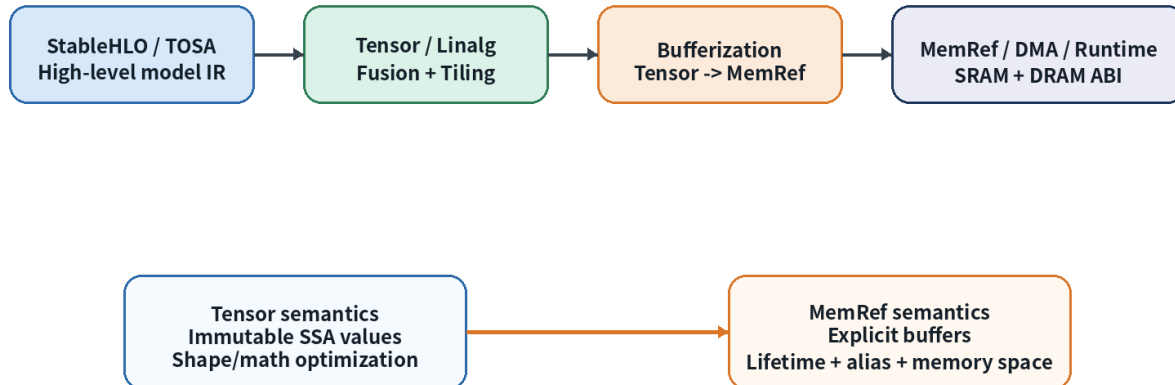


12 강 - Bufferization: tensor -> memref, alias/lifetime 분석

AI/NPU Compiler 를 위한 MLIR 기초 - 20 강 과정 중 12 강

Bufferization Boundary in an NPU Compiler

Do high-level graph/tensor transforms before committing to physical buffers.



1. 강의 목표

- Bufferization 을 tensor semantic 에서 memref/memory planning 으로 넘어가는 compiler boundary 로 이해한다.
- One-Shot Bufferize 의 analysis -> rewrite 구조와 BufferizableOpInterface 의 역할을 설명한다.
- Destination-Passing Style(DPS), outs operand, in-place bufferization, RaW conflict 를 NPU 관점으로 해석한다.
- NPU SRAM/DRAM/accumulator memory planning 을 위한 alias/lifetime 분석 체크리스트를 작성한다.

2. 왜 NPU Compiler 에서 Bufferization 이 중요한가?

NPU compiler 에서 bufferization 은 단순히 자료형을 tensor 에서 memref 로 바꾸는 pass 가 아닙니다. 이 단계 이후부터 compiler 는 실제 storage, copy, lifetime, memory space, runtime ABI 를 책임져야 합니다. 따라서 bufferization 은 성능과 correctness 를 동시에 결정하는 경계입니다.

- tensor 단계: immutable SSA value 로 표현되어 fusion, tiling, canonicalization, algebraic rewrite 가 쉽다.

- memref 단계: mutable buffer reference 가 등장하며 layout, memory space, alias, lifetime, allocation/deallocation 이 드러난다.
- NPU 단계: copy 수, peak live bytes, SRAM tile footprint, DRAM traffic, command buffer descriptor 가 결정된다.

관점	Tensor land	MemRef land	NPU compiler 의미
주요 의미	값 중심 immutable SSA	storage 중심 mutable buffer	abstract tensor 가 physical storage obligation 으로 변환
최적화	fusion, tiling, canonicalization	allocation hoisting, deallocation, copy cleanup	copy traffic 와 SRAM footprint 를 줄이는 단계
오류 위험	semantic mismatch	alias/copy/lifetime 오류	in-place corruption, DMA hazard, ABI mismatch
주요 IR	tensor, linalg-on-tensors	memref, linalg-on-buffers, bufferization	npu_kernel, npu_rt, command buffer

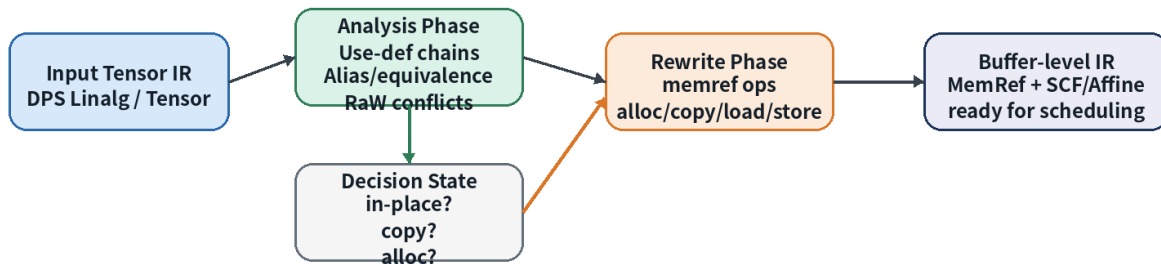
3. 공식 문서 기반 개념 요약

MLIR 문서에서 bufferization 은 tensor semantics 를 가진 op 를 memref semantics 를 가진 op 로 변환하는 과정으로 설명됩니다. One-Shot Bufferize 는 tensor IR 을 memref IR 로 rewrite 하는 핵심 pass 이며, 전체 function 수준의 SSA use-def chain 분석을 통해 in-place 여부와 copy 삽입을 판단합니다.

- One-Shot Bufferize 는 analysis phase 와 rewrite phase 로 나눠 이해하면 좋다.
- analysis phase 는 alias/equivalence set 과 RaW conflict 를 추적한다.
- rewrite phase 는 tensor op 를 memref op, alloc, copy, load/store, subview 등으로 바꾼다.
- custom op 는 BufferizableOpInterface 를 구현해야 bufferization 품질을 통제할 수 있다.

One-Shot Bufferize: Analysis then Rewrite

The best NPU result comes from making buffer decisions with tensor-level SSA information still available.



4. Destination-Passing Style(DPS)

DPS 는 bufferization 품질을 높이는 핵심 형태입니다. 예를 들어 linalg.matmul 은 결과 tensor 를 새로 “생성한 다”고만 표현하지 않고, outs operand 를 통해 result 가 materialize 될 destination 을 제공합니다. 이 destination 이 conflict 없이 재사용 가능하면 compiler 는 result buffer 를 destination buffer 와 alias 시켜 in-place 로 bufferize 할 수 있습니다.

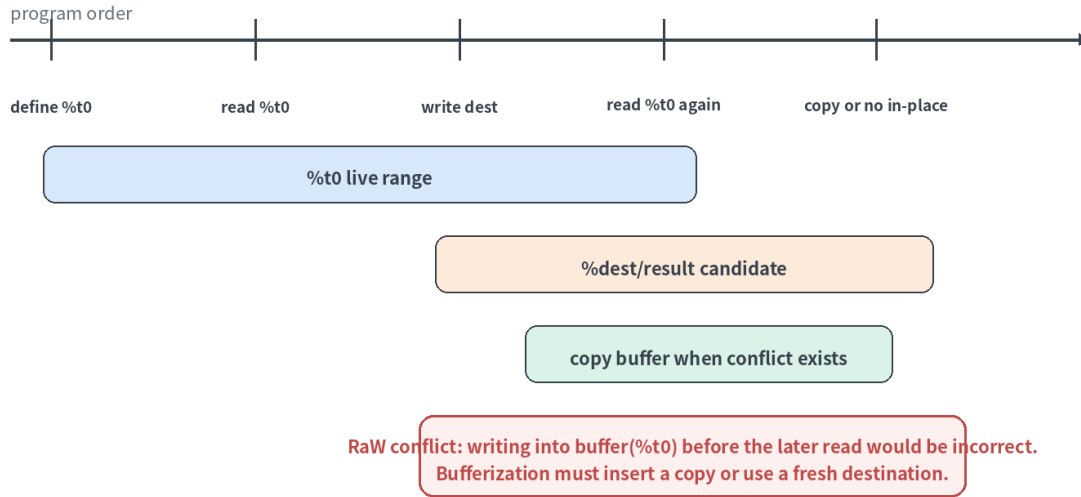
```
func.func @matmul_dps(%A: tensor<128x256xf32>,  
                     %B: tensor<256x128xf32>,  
                     %Cinit: tensor<128x128xf32>) -> tensor<128x128xf32> {  
  %C = linalg.matmul ins(%A, %B : tensor<128x256xf32>, tensor<256x128xf32>)  
    outs(%Cinit : tensor<128x128xf32>) -> tensor<128x128xf32>  
  return %C : tensor<128x128xf32>  
}
```

- NPU 에서는 outs operand 를 output tile buffer 또는 accumulator materialization point 로 해석할 수 있 다.
- DPS 가 잘 유지되면 intermediate copy 가 줄고, SRAM tile buffer 재사용 계획이 단순해진다.
- DPS 를 깨는 composite op 는 bufferization 전에 linalg/tensor primitive 로 낮추는 것이 유리하다.

5. In-place Bufferization 과 RaW Conflict

In-place bufferization 의 기본 질문은 “이 result 를 이미 존재하는 destination buffer 에 써도 안전한가?”입니 다. 안전하지 않은 대표 사례는 read-after-write conflict 입니다. 즉, 어떤 tensor 의 기존 값이 나중에 다시 읽혀야 하는데 같은 buffer 에 먼저 새 값을 써버리면 semantic 이 깨집니다.

Alias / Lifetime Audit for NPU Memory Planning



항목	질문	NPU 위험
Alias	두 SSA value 가 같은 physical buffer/subview 를 가리킬 수 있는가?	동시 DMA/compute 가 같은 SRAM bank 를 덮어쓸 수 있음
Lifetime	각 value 의 last use 는 어디인가?	live range overlap 시 scratch buffer reuse 불가
RaW conflict	나중에 읽을 값을 먼저 overwrite 하는가?	output corruption 또는 nondeterministic result
Copy insertion	copy 가 correctness 를 위해 필요한가?	DRAM/SRAM bandwidth 와 latency 증가

6. bufferization.alloc_tensor 와 boundary op

bufferization.alloc_tensor 는 fresh tensor SSA chain 을 만들기 위한 anchor 로 사용됩니다. 이 op 의 bufferized result 는 다른 buffer 와 alias 하지 않는 새 allocation 으로 취급되므로, 특정 RaW conflict 를 의도적으로 끊을 때 유용합니다. 그러나 NPU 관점에서 fresh allocation 은 SRAM footprint 와 DRAM spill 가능성을 증가시키므로 신중하게 사용해야 합니다.

bufferization.alloc_tensor as a Fresh-Buffer Anchor



Use it sparingly: every fresh allocation may increase SRAM/DRAM pressure, but it can break a correctness conflict.

```
func.func @fresh_anchor(%A: tensor<64x64xf32>, %B: tensor<64x64xf32>) -> tensor<64x64xf32> {  
  // Fresh destination: the result buffer must not alias with A or B.  
  %dst = bufferization.alloc_tensor() : tensor<64x64xf32>  
  %0 = linalg.generic {  
    indexing_maps = [affine_map<((i, j) -> (i, j)), affine_map<((i, j) -> (i, j)), affine_map<((i, j) -> (i, j))],  
    iterator_types = ["parallel", "parallel"]  
  } ins(%A, %B : tensor<64x64xf32>, tensor<64x64xf32>) outs(%dst : tensor<64x64xf32>) {  
    ^bb0(%a: f32, %b: f32, %out: f32):  
      %s = arith.addf %a, %b : f32  
      linalg.yield %s : f32  
  } -> tensor<64x64xf32>  
  return %0 : tensor<64x64xf32>  
}
```

7. Function Boundary 와 Runtime ABI

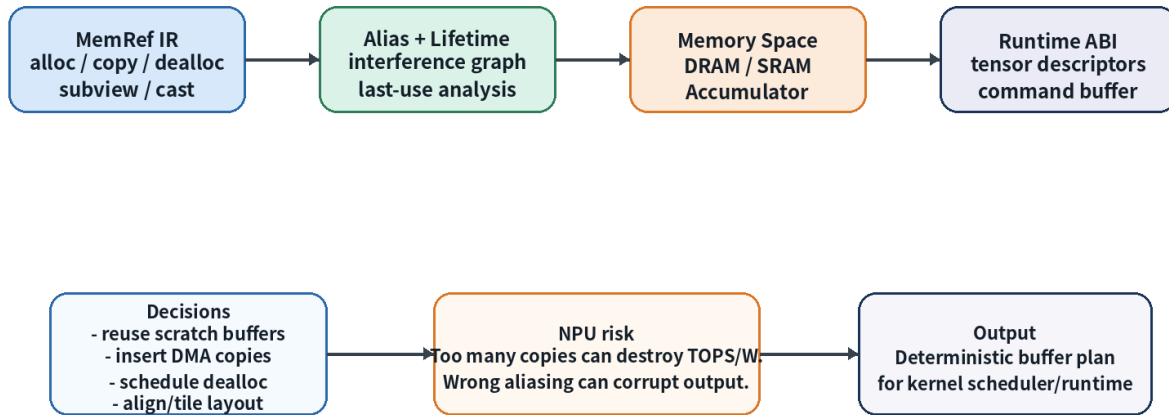
Bufferization 이 function boundary 를 건드리는 순간, NPU runtime ABI 설계와 직접 연결됩니다. tensor argument/result 를 memref argument/result 로 바꾸면, caller/callee 가 같은 descriptor format, layout, memory space, ownership rule 을 공유해야 합니다.

ABI 항목	확인 질문	예시
Descriptor	base pointer, shape, stride, dtype, memory space 가 필요한가?	NPU_TensorDesc {addr, sizes, strides, dtype, space}
Ownership	callee 가 alloc/dealloc 책임을 갖는가?	model output 은 caller-provided buffer 에 materialize
Layout	identity, tiled, blocked, padded layout 중 무엇인가?	NPU-friendly NCHWc / tiled MxN layout
Memory space	DRAM/SRAM/ACCUM 을 ABI 가 표현하는가?	memref<..., 1> as SRAM-like space

8. NPU Memory Planning 연결

From Bufferization to NPU Memory Planning

Bufferization is the compiler boundary where abstract tensors become physical storage obligations.



Bufferization 이후 NPU compiler 는 memref alloc/copy/dealloc, subview, cast, memory_space 정보를 바탕으로 memory planning 을 수행합니다. 이때 중요한 것은 peak live bytes, copy bytes, bank conflict, DMA overlap, descriptor ABI 입니다.

- 동일 tile loop 안에서 lifetime 이 겹치지 않는 scratch buffer 는 storage 를 공유할 수 있다.
- accumulator buffer 는 dtype 과 lifetime 이 activation buffer 와 다르므로 별도 memory class 로 보는 것이 안전하다.
- DRAM<->SRAM DMA buffer 는 double buffering 때문에 같은 logical tensor 도 ping/pong physical buffer 를 가질 수 있다.
- memref memory_space 는 target-specific attribute 이므로 NPU backend 에서 명확한 enum/attribute contract 를 정의한다.

9. 권장 Pass Pipeline

```
# Conceptual NPU compiler pipeline around bufferization
builtin.module(
  func.func(canonicalize, cse),
  stablehlo-legalize-to-linalg-or-tosa,
  npu-graph-fuse-matmul-epilogue,
  npu-graph-layout-propagation,
  linalg-tile-and-fuse-on-tensors,
  one-shot-bufferize{bufferize-function-boundaries=true},
  ownership-based-buffer-deallocation,
  func.func(canonicalize, cse),
  npu-assign-memory-space,
  npu-plan-scratch-buffers,
```

npu-lower-memref-to-dma-and-runtime
)

- 핵심 원칙: high-level fusion/tiling/layout decision 은 가능한 한 tensor land 에서 끝낸다.
- one-shot-bufferize 이전에 bufferizable dialect model 이 등록되어야 한다.
- bufferization 이후에는 copy/debug/ownership/memory-space cleanup pass 를 반드시 둔다.

10. 실습 목표

실습	내용	산출물
A	linalg-on-tensors MatMul+Bias+ReLU 의 destination operand 표시	buffer(result) 후보 표
B	pseudo one-shot-bufferize 전후 IR 비교	alloc/copy/dealloc count + copy 이유
C	tile buffer lifetime chart 작성	DRAM/SRAM/ACCUM memory- space plan
D	FileCheck regression 작성	unexpected memref.copy 방지 test

11. Takeaways

- Bufferization 은 tensor semantic 을 memref storage semantic 으로 바꾸는 compiler boundary 다.
- One-Shot Bufferize 는 SSA use-def chain 분석을 통해 in-place/copy 결정을 수행한다.
- DPS/linalg-on-tensors 형태를 유지하면 high-quality bufferization 가능성이 높아진다.
- NPU compiler 는 bufferization output 을 alias/lifetime/memory-space/runtime ABI 계획으로 연결해야 한다.

12. References

- MLIR Bufferization: <https://mlir.llvm.org/docs/Bufferization/>
- MLIR Bufferization Dialect: <https://mlir.llvm.org/docs/Dialects/BufferizationOps/>
- MLIR MemRef Dialect: <https://mlir.llvm.org/docs/Dialects/MemRef/>
- MLIR Builtin Dialect - MemRef type and memory space:
<https://mlir.llvm.org/docs/Dialects/Builtin/>
- MLIR Tensor Dialect: <https://mlir.llvm.org/docs/Dialects/TensorOps/>
- MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>